Exp: 12 **End-to-end Communication at the Transport Layer**

Aim:

To implement (a) an echo client-server using TCP/UDP sockets and (b) a chat program using socket programming.

Requirements:

* Python 3 (or C / Java)
* Two terminals

Procedure:

1. Run the server program in one terminal.
2. Run the client program in another terminal and send messages.
3. Observe the echo (or chat messages) on both sides. Use the server's IP instead of 127.0.0.1 to run on two machines.

(a) TCP Echo Server:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 5000)); s.listen(1)
print("TCP echo server listening on port 5000")
conn, addr = s.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data: break
    print("Received:", data.decode())
    conn.sendall(data)                    # echo back
conn.close(); s.close()
```

TCP Echo Client:

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 5000))
while True:
    msg = input("Enter message (quit to exit): ")
    if msg == "quit": break
    c.sendall(msg.encode())
    print("Echo from server:", c.recv(1024).decode())
c.close()
```

UDP Echo Server:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("0.0.0.0", 6000))
print("UDP echo server listening on port 6000")
while True:
    data, addr = s.recvfrom(1024)
    print("Received from", addr, ":", data.decode())
    s.sendto(data, addr)
```

UDP Echo Client:

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    msg = input("Enter message (quit to exit): ")
    if msg == "quit": break
    c.sendto(msg.encode(), ("127.0.0.1", 6000))
    data, _ = c.recvfrom(1024)
    print("Echo from server:", data.decode())
c.close()
```

(b) Chat Server (tcp):

```
import socket, threading
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 7000)); s.listen(1)
print("Chat server waiting on port 7000 ...")
conn, addr = s.accept()
```

```
print("Client connected:", addr)
def receive():
    while True:
        data = conn.recv(1024)
        if not data: print("Client disconnected"); break
        print("\nClient:", data.decode())
threading.Thread(target=receive, daemon=True).start()
while True:
    msg = input("")
    conn.sendall(msg.encode())
    if msg.lower() == "bye": break
conn.close(); s.close()
```

Chat Client (tcp):

```
import socket, threading
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 7000))
print("Connected to chat server. Type 'bye' to exit.")
def receive():
    while True:
        data = c.recv(1024)
        if not data: ~~break~~ break
        print("\nServer:", data.decode())
threading.Thread(target=receive, daemon=True).start()
while True:
    msg = input("")
    c.sendall(msg.encode())
    if msg.lower() == "bye": break
c.close()
```

Result:

Echo client-server programs using TCP and UDP sockets and a chat program were implemented successfully.